

Case-Based Scenario: Detecting Network Intrusions with Machine Learning

Background

It is a routine Tuesday shift for Meera, a security analyst at a mid-sized company's Security Operations Center (SOC). Her manager has just asked her a simple but high-stakes question: "Can we catch an attack before it causes damage, using data rather than a list of known signatures?" Meera's team currently relies on a legacy intrusion detection system that flags traffic only when it matches a known attack signature. It works well for familiar threats, but it missed a new denial-of-service variant just weeks ago, and it required hours of manual log review before anyone noticed. Meera has access to a large, labeled history of network connections — the NSL-KDD dataset — recording over 148,000 connections along with details like protocol type, service, connection duration, byte counts, and error rates. She knows that analysts typically rely on instinct and known red flags: a spike in traffic, an unfamiliar service, a string of failed logins. But no analyst can consistently weigh dozens of such signals by hand across thousands of connections a day.

The Approach

Meera turns her manager's question into a machine learning problem. She treats "Is this connection normal or an attack?" as a classification task, using the connection's features to predict whether it is benign or malicious, and if malicious, which of four attack families it falls into: Denial of Service (DoS), Probing, Remote-to-Local (R2L), or User-to-Root (U2R). She engineers and selects features that reflect how a human analyst would naturally reason about a connection: the protocol and service being used, how long the connection lasted, how many bytes were sent in each direction, how often connections to the same host or service resulted in errors, and whether a user was logged in. She then builds two predictive models to answer her manager's question. The first, Random Forest, trains many decision trees on random slices of the data and combines their votes, giving a stable, hard-to-fool baseline. The second, Gradient-Boosted Trees, builds trees one after another, with each new tree correcting the mistakes of the ones before it, which often captures subtler, non-obvious combinations of features that a single tree or a simple vote would miss.

The Outcome

After training both models on past traffic and testing them on connections they have never seen — including attack types absent from training, deliberately included to make the test realistic — Meera compares their predictions against the true labels. Both models perform very well on the two most common attack types, correctly catching nearly all DoS and Probe attacks. But test accuracy drops well below the near-perfect scores seen during validation, a humbling but honest outcome. On closer analysis, she finds that both models catch only a small fraction of R2L attacks and almost none of the rare U2R attacks, precisely the categories with the fewest training examples. Gradient-Boosted Trees edges out Random Forest overall, but neither model can be trusted on the rarest attack types without more examples to learn from — a useful, if less flattering, insight for her manager.

Relevance to the Project

This scenario reflects the real-world motivation behind the project: network intrusion detection, like many rare-event prediction problems, resists a perfect solution, and the value of a machine learning approach lies not in flawless detection, but in offering a systematic, behavior-based way to flag suspicious traffic where fixed signatures and human review alone fall short. The project's use of Random Forest and Gradient-Boosted Trees, applied to connection-level features such as protocol type, service, byte counts, and error rates, and evaluated with Accuracy, Precision, Recall, F1-score, the Confusion Matrix, and ROC-AUC, is a direct implementation of the approach Meera follows in this scenario